

Junior AI Developer Assignment

RAG-based Document Q&A System with Analytics Dashboard

1. Overview

In this assignment, you will build a Retrieval-Augmented Generation (RAG) system that answers questions about a provided document, expose it through a FastAPI backend with SQL-based usage logging, and build a simple frontend to interact with it.

This task is designed to evaluate your practical skills in Python, Generative AI / LLM integration, SQL, backend API design, and your ability to ship a complete, working, well-documented project.

2. What You're Given

You will be provided with one (1) PDF document: the AWS Customer Agreement.

This is the only data source you need. The entire assignment - RAG, SQL logging, and the dashboard - is built around this single document plus the usage logs your own system generates while you test it.

3. What You Need to Build

3.1 RAG Pipeline (Core)

- Parse and chunk the provided PDF (justify your chunk size and overlap choice in the README).
- Generate embeddings for each chunk using any library of your choice (sentence-transformers, OpenAI embeddings, HuggingFace, etc.).
- Store embeddings in a vector store (FAISS, ChromaDB, etc. no need to host one).
- On receiving a user query: retrieve the most relevant chunks (top-k, your choice of k), construct a prompt with the retrieved context, and call an LLM (OpenAI API, HuggingFace Inference API, or a local model via Ollama - your choice) to generate the answer.
- Return the generated answer along with the source chunk(s)/snippet used to answer.
- Handle the case where the answer is not present in the document (the model should say so rather than hallucinate).

3.2 SQL Logging & Analytics

Every call to your question-answering endpoint must be logged into a SQL database (SQLite is sufficient or you can use PostgreSQL). You need to think what will be the schema of the table.

Important: Since you only have one static document, the SQL part of this task is about usage analytics, not document content. After your pipeline is working, run at least 30 test queries against your own endpoint a mix of genuinely answerable questions and a few irrelevant/out-of-scope ones - so your logs table has realistic data to query.

Expose at least the following via an analytics endpoint, backed by SQL queries (GROUP BY / COUNT / AVG, etc.):

- Most frequently asked questions
- Queries where no answer was found in context
- Average response latency

3.3 FastAPI Backend

- POST /ingest - processes and embeds the provided PDF
- POST /ask - accepts a query, runs the RAG pipeline, returns the answer + source + logs the interaction
- GET /analytics - runs the SQL analytics queries and returns the results as JSON
- Use Pydantic models for request/response validation.
- Handle edge cases gracefully: empty query, no document ingested yet, etc. (return meaningful HTTP error codes/messages, not raw stack traces).

3.4 Frontend Dashboard

Build a simple frontend that talks to your FastAPI backend. You may use either:

- Streamlit (using the requests library to call your FastAPI endpoints), or
- React / Next.js

Both are equally acceptable - we are evaluating functional integration and clarity, not visual design polish. Your frontend must include:

- A simple chat-style interface to ask a question and view the generated answer (with source).
- An analytics view showing the data from the /analytics endpoint (a table or simple chart is fine).

Note: If using Streamlit, run it as a separate process from FastAPI (Streamlit calling FastAPI over HTTP) -document both startup commands clearly in your README.

4. Deliverables

1. A public GitHub repository containing your full source code.
2. A README.md file that includes:
 - Setup and run instructions that work end-to-end (someone should be able to clone your repo and run it using only your README)

- Architecture overview (how the pieces fit together)
- Key design decisions and assumptions (chunking strategy, top-k, model/provider choice, etc.)
- 3. A short demo video or GIF (2-3 minutes) showing the app running - asking a question and viewing the analytics dashboard.

5. Evaluation Criteria

Category	Weight	What We're Looking For
RAG Pipeline Design	20%	Chunking strategy, retrieval relevance (top-k), reasoning behind design choices - not just "it runs"
FastAPI Backend & SQL	20%	Endpoint design, Pydantic validation, error handling, correctness of SQL logging schema and analytics queries
Code Quality & Structure	20%	Project organization, modularity, naming, documented code (docstrings/comments), no spaghetti logic
Frontend Integration	15%	Functional Streamlit or React/Next.js UI that correctly consumes the FastAPI endpoints; clarity over visual polish
Documentation (README)	15%	Setup instructions actually work end-to-end, architecture explained, assumptions/design decisions justified
Demo & Submission Hygiene	10%	Working demo video/GIF, clean public GitHub repo

6. Logistics

- Time expectation: You will have 48 hours to submit.
- Cost: You are not required to use a paid API. Free-tier or local model options (HuggingFace Inference API, sentence-transformers + a free LLM endpoint, Ollama, etc.) are completely acceptable.

Note on Originality & Plagiarism

All code, prompts, documentation, and analysis must be your own work. You are encouraged to consult public references and official documentation, but copy-pasting from ChatGPT, GitHub, blogs, or other sources without understanding or attribution is strictly prohibited.

We will conduct manual code reviews and may ask you to walk through your implementation and design choices. Any detected plagiarism will result in immediate disqualification.